

EN UNA FRASE Sabes relacionar entidades JPA con `@ManyToOne` y `@JoinColumn`, cargar las relaciones con `JOIN FETCH` para evitar el problema N+1 y exponer los préstamos por REST con un DTO en lugar de la entidad.

LO ESENCIAL

<code>@ManyToOne</code>	Muchos registros de esta tabla apuntan a uno de otra: muchos <code>Prestamo</code> para un <code>Usuario</code> (o un <code>Libro</code>)
<code>@JoinColumn(name = "usuario_id", nullable = false)</code>	Nombre de la columna de clave foránea en la tabla <code>prestamo</code> ; sin ella Hibernate genera un nombre automático
<code>FetchType.LAZY</code>	Carga la relación solo cuando llamas al getter. Es lo recomendado
<code>FetchType.EAGER</code>	Carga la relación siempre con un JOIN automático. Es el valor por defecto de <code>@ManyToOne</code> : por eso escribes <code>LAZY</code> a mano
Problema N+1	Cargas N préstamos y accedes a sus relaciones LAZY en un bucle: 2N+1 consultas en vez de una
<code>JOIN FETCH</code>	Instrucción JPQL que trae las entidades relacionadas en el mismo SELECT
<code>@Query</code>	Escribe la consulta JPQL en el propio método del repositorio
DTO (<code>PrestamoRespuesta</code>)	Aplana los datos de la respuesta y aísla el modelo interno; evita la <code>LazyInitializationException</code>
record como DTO	Inmutable, con getters automáticos; Jackson lo serializa sin configuración
<code>@Transactional</code>	Todas las operaciones del método se completan o ninguna (rollback automático); a nivel de clase cubre todos los métodos
<code>@Transactional(readOnly = true)</code>	Marca los métodos de solo lectura y permite optimizaciones a Hibernate
<code>@PatchMapping</code>	Verbo PATCH para actualizar solo una parte del recurso (la <code>fechaDevolucion</code>)
<code>@Column(nullable, unique, length, name)</code>	Restricciones de la columna en la base de datos; <code>nullable = false</code> complementa a <code>@NotNull</code> , no lo sustituye
<code>fechaDevolucion a null</code>	Así se representa un préstamo activo

ERRORES TÍPICOS

- 1 `LazyInitializationException ... no Session` al devolver `List<Prestamo>`**
✓ la sesión JPA ya está cerrada cuando Jackson llama a `getUsuario()`. Devuelve `PrestamoRespuesta` con `.map(PrestamoRespuesta::desde)`.
- 2 Un fallo a mitad de una operación deja los datos a medias, sin rollback**
✓ falta `@Transactional` en el servicio. Sin ella, cada operación del repositorio va en su propia transacción.
- 3 Los logs muestran JOINS con libro y usuario que no has pedido**
✓ `@ManyToOne` sin `fetch` es `EAGER`. Pon `fetch = FetchType.LAZY` y usa `JOIN FETCH` solo donde haga falta.
- 4 `Referential integrity constraint violation` al arrancar**
✓ `data.sql` referencia ids que no existen. Inserta primero libros y usuarios, después préstamos, y usa los ids que genera `IDENTITY` (1, 2, 3...).
- 5 `405 Method Not Allowed` con `Allow: PATCH`**
✓ has llamado a la devolución con PUT. El endpoint es `PATCH /prestamos/{id}/devolucion`.

EN EL PROYECTO BIBLIOTECA

Llegan las entidades `Usuario` y `Prestamo`, sus repositorios (`UsuarioJpaRepository`, `PrestamoJpaRepository`), `PrestamoService`, `PrestamoController` y el DTO `PrestamoRespuesta`. `ManejadorGlobalErrores` traduce ahora tres «no encontrado» a 404 (libro, usuario y préstamo, con `UsuarioNoEncontradoException` y `PrestamoNoEncontradoException` nuevas) y `data.sql` añade tres usuarios y tres préstamos: dos activos y uno devuelto.

■ CÓDIGO MÍNIMO Y PETICIONES

Las dos relaciones de `Prestamo`, siempre con `LAZY` explícito:

```
@Entity
@Table(name = "prestamo")
public class Prestamo {

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "usuario_id", nullable = false)
    private Usuario usuario;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "libro_id", nullable = false)
    private Libro libro;

    private LocalDate fechaPrestamo;
    private LocalDate fechaDevolucion; // null = préstamo activo
}
```

JAVA

La consulta de `PrestamoJpaRepository` que trae usuario y libro en un solo SELECT:

```
@Query("""
    SELECT p FROM Prestamo p JOIN FETCH p.usuario JOIN FETCH p.libro
    WHERE p.fechaDevolucion IS NULL
    """)
List<Prestamo> buscarActivosConUsuarioYLibro();
```

JAVA

El controlador devuelve el DTO, nunca la entidad:

```
@GetMapping
public List<PrestamoRespuesta> listarTodos() {
    return service.obtenerTodos().stream().map(PrestamoRespuesta::desde).toList();
}
```

JAVA

```
### Listar todos los préstamos activos
GET http://localhost:8080/prestamos

### Préstamos activos del usuario con id 1
GET http://localhost:8080/prestamos/usuario/1

### Registrar nuevo préstamo: usuario 2 pide el libro 3
POST http://localhost:8080/prestamos/usuario/2/libro/3

### Registrar devolución del préstamo con id 1
PATCH http://localhost:8080/prestamos/1/devolucion
```

HTTP